

SET-1 ANSWER KEY

BATCH-2

- 1.) i) Creating a dataframe from dictionary of ndarray/lists perform basic operations on rows/columns like selecting, deleting, adding, and renaming using pandas.
- ii) Write a pandas program to sort the dataframe first by 'name' in descending order, then by 'score' in ascending order.

Ans:- i.) **AIM**

To create a DataFrame from dictionary of lists/ndarray and perform basic operations like selecting, deleting, adding, and renaming rows/columns using pandas.

ALGORITHM

1. Import pandas library.
2. Create a dictionary and convert it into a DataFrame.
3. Select required rows/columns from the DataFrame.
4. Add, delete, and rename columns.
5. Display the modified DataFrame.

PROGRAM

```
import pandas as pd

# Creating DataFrame
data = {
    'Name': ['A', 'B', 'C'],
    'Age': [20, 21, 19],
    'Marks': [85, 90, 88]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Selecting column
print("\nSelecting Name column:")
print(df['Name'])

# Adding new column
df['Grade'] = ['A', 'A+', 'A']
print("\nAfter Adding Grade column:")
```

```
print(df)
```

```
# Deleting a column
```

```
df.drop('Age', axis=1, inplace=True)
```

```
print("\nAfter Deleting Age column:")
```

```
print(df)
```

```
# Renaming column
```

```
df.rename(columns={'Marks': 'Score'}, inplace=True)
```

```
print("\nAfter Renaming column:")
```

```
print(df)
```

OUTPUT:-

Original DataFrame:

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	88

Selecting Name column:

0	A
1	B

2 C

After Adding Grade column:

	Name	Age	Marks	Grade
0	A	20	85	A
1	B	21	90	A+
2	C	19	88	A

After Deleting Age column:

	Name	Marks	Grade
0	A	85	A
1	B	90	A+
2	C	88	A

After Renaming column:

	Name	Score	Grade
0	A	85	A
1	B	90	A+
2	C	88	A

RESULT

Thus, the DataFrame was created and basic operations like selecting, adding, deleting, and renaming were performed successfully.

ii.) AIM

To write a pandas program to sort the DataFrame by 'name' in descending order and 'score' in ascending order.

ALGORITHM

1. Import pandas library.
2. Create a DataFrame with name and score columns.
3. Use sort_values() function.
4. Sort by name (descending) and score (ascending).
5. Display the sorted DataFrame.

PROGRAM

```
import pandas as pd

# Creating DataFrame
data = {
    'name': ['John', 'Alice', 'Bob', 'John'],
    'score': [85, 90, 85, 80]
}
```

```
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Sorting
sorted_df = df.sort_values(by=['name', 'score'],
                           ascending=[False, True])

print("\nSorted DataFrame:")
print(sorted_df)
```

OUTPUT

Original DataFrame:

	name	score
0	John	85
1	Alice	90
2	Bob	85
3	John	80

Sorted DataFrame:

	name	score
--	------	-------

3	John	80
0	John	85
2	Bob	85
1	Alice	90

RESULT

Thus, the DataFrame was successfully sorted by name in descending order and score in ascending order.

2.) i) Write a pandas program to select the rows where the number of attempts in the examination is greater than 2.

ii) Write a pandas program to append a new row 'k' to dataframe with given values for each column. Now delete the new row and return the original dataframe.

Ans:- i.) **AIM**

To write a pandas program to select rows where the number of attempts is greater than 2.

ALGORITHM

1. Import pandas library.

2. Create a DataFrame with attempts column.
3. Apply condition to filter rows (attempts > 2).
4. Store filtered data in a new DataFrame.
5. Display the result.

PROGRAM

```
import pandas as pd

# Creating DataFrame
data = {
    'name': ['A', 'B', 'C', 'D'],
    'attempts': [1, 3, 2, 4],
    'score': [80, 90, 85, 88]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Selecting rows where attempts > 2
result = df[df['attempts'] > 2]
```



```
print("\nRows where attempts > 2:")  
print(result)
```

OUTPUT

Original DataFrame:

	name	attempts	score
0	A	1	80
1	B	3	90
2	C	2	85
3	D	4	88

Rows where attempts > 2:

	name	attempts	score
1	B	3	90
3	D	4	88

RESULT

Thus, rows where the number of attempts is greater than 2 were successfully selected.

ii.) **AIM**

To write a pandas program to append a new row 'k' to the DataFrame and then delete it to return the original DataFrame.

ALGORITHM

1. Import pandas library.
2. Create a DataFrame.
3. Append a new row using loc or concat.
4. Delete the appended row.
5. Display the final DataFrame.

PROGRAM

```
import pandas as pd

# Creating DataFrame
data = {
    'name': ['A', 'B', 'C'],
    'score': [80, 90, 85]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
```

```
print(df)

# Appending new row 'k'
df.loc['k'] = ['D', 88]

print("\nAfter Appending row 'k':")
print(df)

# Deleting row 'k'
df = df.drop('k')

print("\nAfter Deleting row 'k':")
print(df)
```

OUTPUT

Original DataFrame:

	name	score
0	A	80
1	B	90
2	C	85

After Appending row 'k':

	name	score
0	A	80
1	B	90
2	C	85
k	D	88

After Deleting row 'k':

	name	score
0	A	80
1	B	90
2	C	85

RESULT

Thus, a new row was appended and successfully deleted, restoring the original DataFrame.

- 3.) i) Write a python program to change the vertical spacing between legend entries in Matplotlib?
- ii) Write a python program to change the color of a graph plot in matplotlib using dataset csv.

Ans:- i.) AIM

To write a Python program to change the vertical spacing between legend entries in Matplotlib.

ALGORITHM

1. Import matplotlib library.
2. Create sample data for plotting.
3. Plot multiple lines with labels.
4. Use legend() with labelspacing parameter.
5. Display the graph.

PROGRAM

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4]
```

```
y1 = [10, 20, 25, 30]
```

```
y2 = [15, 25, 20, 35]
```

```
# Plotting
```

```
plt.plot(x, y1, label='Line 1')
```

```
plt.plot(x, y2, label='Line 2')
```

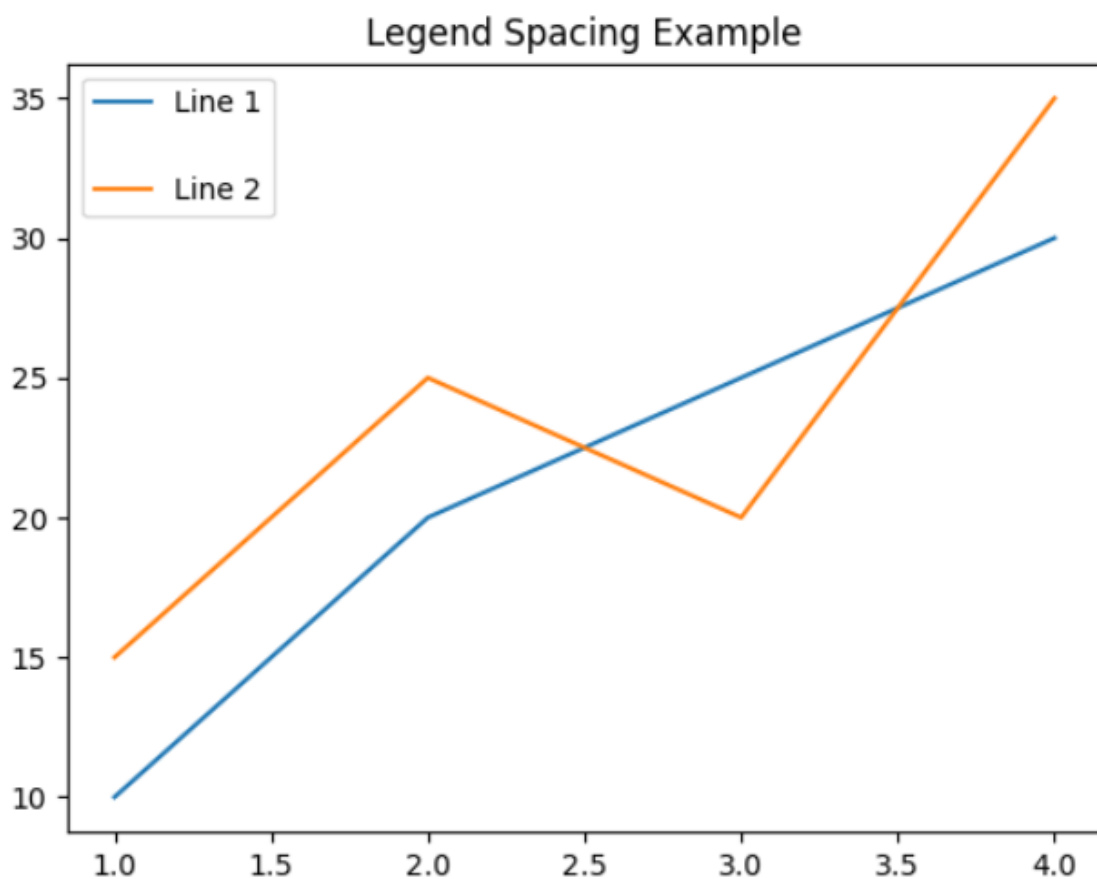
```
# Legend with vertical spacing
```

```
plt.legend(labelspace=2)
```

```
plt.title("Legend Spacing Example")
```

```
plt.show()
```

OUTPUT



RESULT

Thus, the vertical spacing between legend entries was successfully changed using the labelspace parameter.

ii.) AIM

To write a Python program to change the color of a graph plot in Matplotlib using data from a CSV file.

ALGORITHM

- 1. Import pandas and matplotlib libraries.**
- 2. Read data from CSV file.**
- 3. Extract required columns.**
- 4. Plot graph using specified color.**
- 5. Display the graph.**

PROGRAM

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
# Creating sample dataset and saving to CSV  
data = {  
    'x': [1, 2, 3, 4, 5],  
    'y': [10, 20, 15, 25, 30]  
}  
  
df = pd.DataFrame(data)
```

```
df.to_csv('data.csv', index=False)
```

```
# Reading CSV file
```

```
df = pd.read_csv('data.csv')
```

```
x = df['x']
```

```
y = df['y']
```

```
# Plot with color
```

```
plt.plot(x, y, color='green')
```

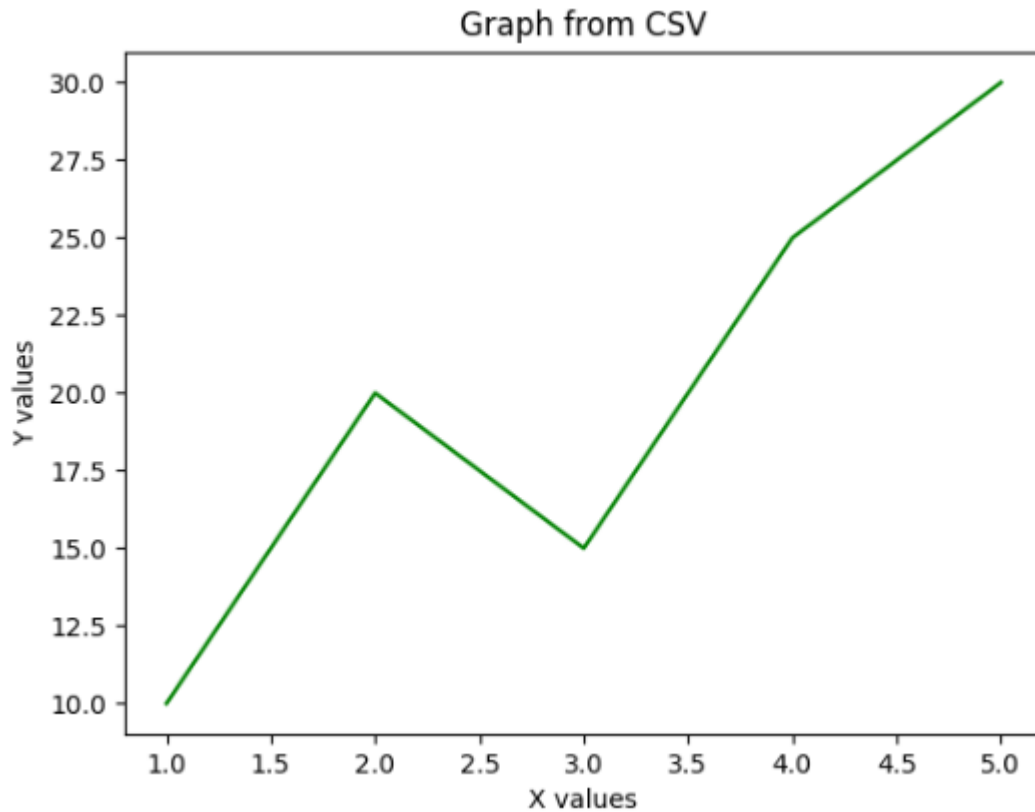
```
plt.title("Graph from CSV")
```

```
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
```

```
plt.show()
```

OUTPUT



RESULT

Thus, the graph was successfully plotted using CSV data with the specified color.

- 4.) Create a figure of size 15 x 8 with two subplots, top and bottom. Draw two lines in the top axes, one green and one orange. Add a legend for the top plot, Green and Orange. Put this legend in the topmiddle of graph using matplotlib.**

Ans:-

AIM

To create a figure of size 15×8 with two subplots and draw two lines in the top subplot with a legend positioned at the top middle.

ALGORITHM

- 1. Import matplotlib library.**
- 2. Create a figure with two subplots (top and bottom).**
- 3. Plot two lines in the top subplot with different colors.**
- 4. Add legend and position it at the top middle.**
- 5. Display the figure.**

PROGRAM

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4]
```

```
y1 = [10, 20, 15, 25]
```

```
y2 = [5, 15, 10, 20]
```

```
# Create figure with 2 subplots
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8))
```

Top subplot

ax1.plot(x, y1, color='green', label='Green')

ax1.plot(x, y2, color='orange', label='Orange')

Legend at top middle

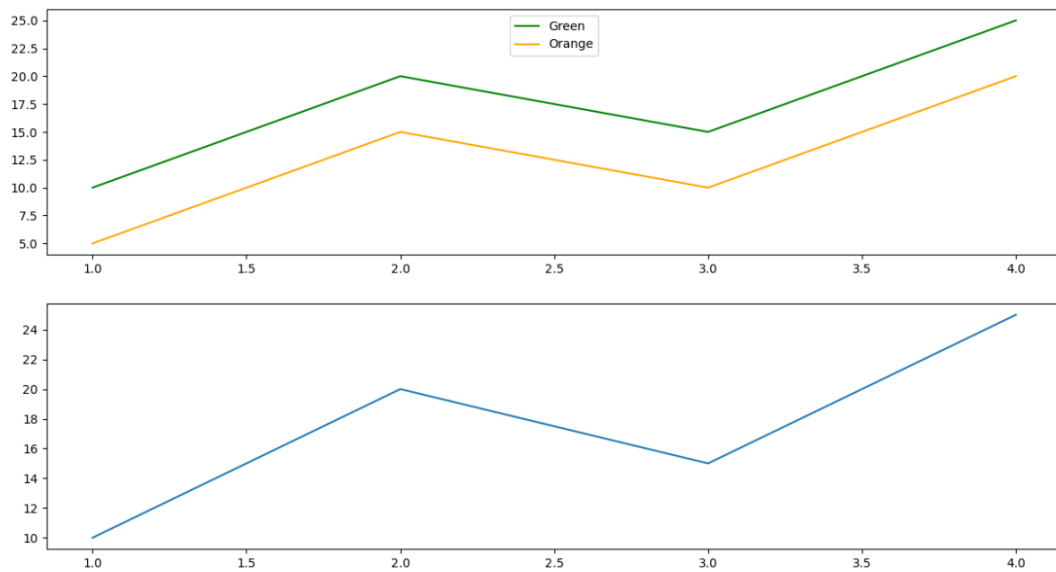
ax1.legend(loc='upper center')

Bottom subplot (empty or optional)

ax2.plot(x, y1)

plt.show()

OUTPUT



RESULT

Thus, the figure with two subplots was created and the legend was successfully positioned at the top middle of the graph.

- 5.) Draw the frequency distribution table and frequency distribution curve for the following data:
2, 3, 1, 4, 2, 2, 3, 1, 4, 4, 4, 2, 2, 2.**

Ans:- AIM

To draw the frequency distribution table and frequency distribution curve for the given data.

ALGORITHM

- 1. Input the given data values.**
- 2. Count the frequency of each unique value.**
- 3. Create a frequency distribution table.**
- 4. Plot the frequency distribution curve using matplotlib.**
- 5. Display the graph.**

PROGRAM

```
import matplotlib.pyplot as plt
```

Given data

data = [2, 3, 1, 4, 2, 2, 3, 1, 4, 4, 4, 2, 2, 2]

Finding frequency

freq = {}

for i in data:

freq[i] = freq.get(i, 0) + 1

Sorting values

x = sorted(freq.keys())

y = [freq[i] for i in x]

Display frequency table

print("Value\tFrequency")

for i in x:

print(i, "\t", freq[i])

Plotting frequency curve

plt.plot(x, y, marker='o')

plt.title("Frequency Distribution Curve")

plt.xlabel("Values")

```
plt.ylabel("Frequency")
```

```
plt.show()
```

OUTPUT

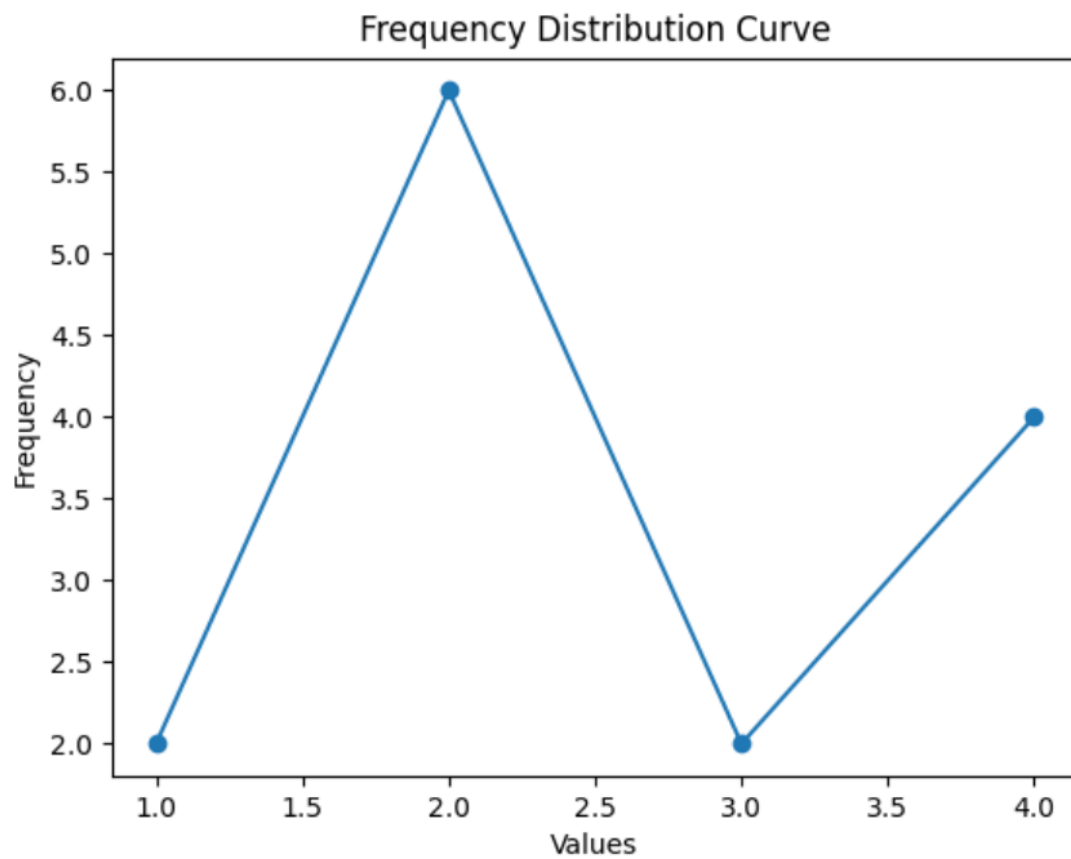
Value	Frequency
-------	-----------

1	2
---	---

2	6
---	---

3	2
---	---

4	4
---	---



RESULT

Thus, the frequency distribution table and curve were successfully constructed for the given data.

- 6.) The table below gives the values of runs scored by Virat Kohli in last 25 T-20 matches. Represent the data in the form of less than type cumulative frequency distribution:-

45	34	50	75	22
56	63	70	49	33
0	8	14	39	86
92	88	70	56	50
57	45	42	12	39

Draw the cumulative frequency distribution table in the form of cumulative frequency distribution line curve

Ans:- AIM

To construct the less than type cumulative frequency distribution table and draw its cumulative frequency curve for the given data.

ALGORITHM

- 1. Input and sort the given data.**
- 2. Form class intervals and calculate frequencies.**
- 3. Compute cumulative frequencies (less than type).**
- 4. Take upper class limits and cumulative frequencies.**
- 5. Plot the cumulative frequency curve.**

PROGRAM

```
import matplotlib.pyplot as plt
```

```
# Given data
```

```
data = [45, 34, 50, 75, 22,  
        56, 63, 70, 49, 33,  
        0, 8, 14, 39, 86,  
        92, 88, 70, 56, 50,  
        57, 45, 42, 12, 39]
```

```
# Sorting data
```

```
data.sort()
```

```
# Class intervals
```

```
classes = [(0,20), (20,40), (40,60), (60,80), (80,100)]
```

```
freq = []
```



```

# Calculate frequency
for c in classes:
    count = 0
    for x in data:
        if c[0] <= x < c[1]:
            count += 1
    freq.append(count)

# Calculate cumulative frequency
cf = []
total = 0
for f in freq:
    total += f
    cf.append(total)

# Display table
print("Class Interval\tFrequency\tCumulative Frequency")
for i in range(len(classes)):
    print(f"{classes[i][0]}-{classes[i][1]}\t\t{freq[i]}\t\t{cf[i]}")

# Upper class limits
upper_limits = [c[1] for c in classes]

# Plot cumulative frequency curve
plt.plot(upper_limits, cf, marker='o')

plt.title("Less Than Cumulative Frequency Curve")
plt.xlabel("Upper Class Limits")

```

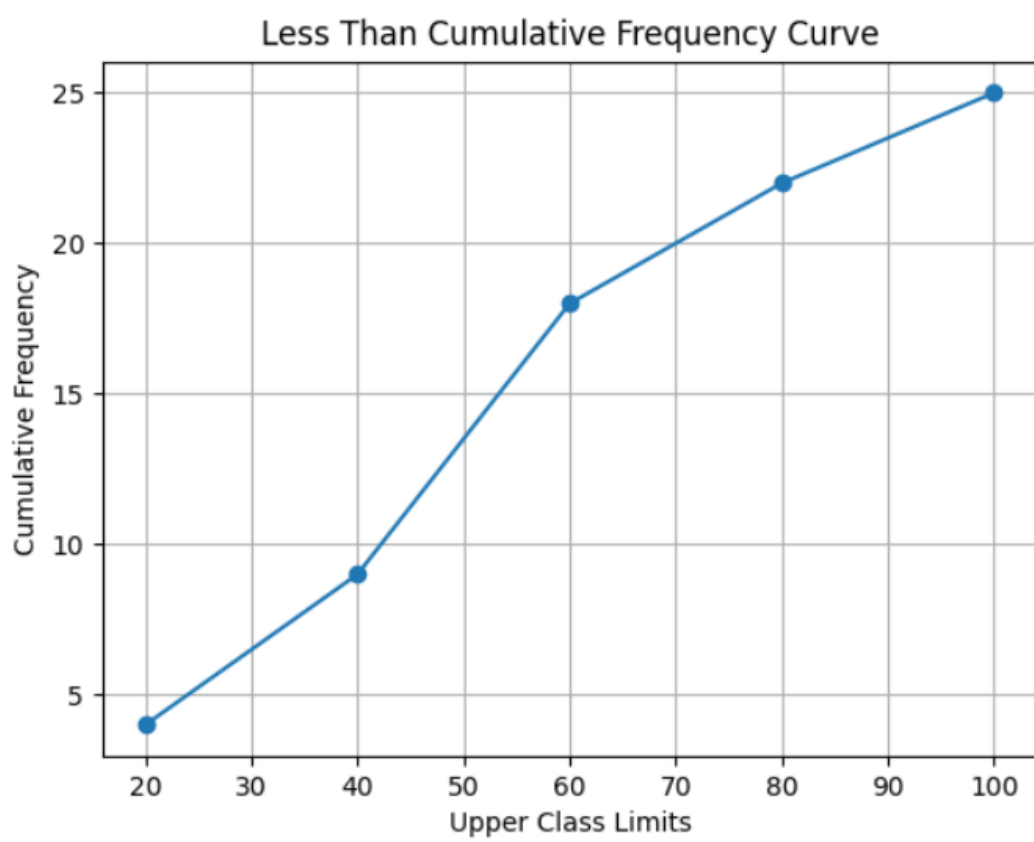
```
plt.ylabel("Cumulative Frequency")
```

```
plt.grid()
```

```
plt.show()
```

OUTPUT

Class Interval	Frequency	Cumulative Frequency
0-20	4	4
20-40	5	9
40-60	9	18
60-80	4	22
80-100	3	25



RESULT

Thus, the less than type cumulative frequency distribution table and its curve were successfully constructed.

- 7.) Write a python program calculate the probability under a normal curve for normal distribution in statistics.

Ans:- **AIM**

To write a Python program to calculate the probability under a normal curve for a normal distribution.

ALGORITHM

1. Import required libraries.
2. Define mean and standard deviation.
3. Convert given value to standard normal variable (Z-score).
4. Use normal distribution function to find probability.
5. Display the result.

PROGRAM

```
from scipy.stats import norm
```

```
# Given values
```

```
mean = 50
```

```
std_dev = 10
```

```
x = 60 # value
```

```
# Calculate probability  $P(X \leq x)$ 
```

```
probability = norm.cdf(x, mean, std_dev)
```

```
print("Probability =", probability)
```

OUTPUT

Probability = 0.8413447460685429

RESULT

Thus, the probability under the normal curve for the given distribution was successfully calculated.

- 8.) Analysis Correlation and scatterplot is a type of graph with data points representing a relationship between two quantities. For example, the following table contains points on a graph:

x	y
1	2
1.5	3
2	5
2.5	7
3	10
3.5	12
4	13
4.5	14
5	18
5.5	20

On a graph, the table would be plotted as (x,y) points with the first column being the x values and the second column being the y values.

Ans:- **AIM**

To analyze correlation and draw a scatter plot representing the relationship between two quantities.

ALGORITHM

1. Import required libraries.
2. Define x and y data points.
3. Plot scatter graph using matplotlib.
4. Label axes and add title.

5. Display the graph.

PROGRAM

```
import matplotlib.pyplot as plt
```

```
# Given data
```

```
x = [1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5]
```

```
y = [2, 3, 5, 7, 10, 12, 13, 14, 18, 20]
```

```
# Scatter plot
```

```
plt.scatter(x, y)
```

```
plt.title("Scatter Plot for Correlation Analysis")
```

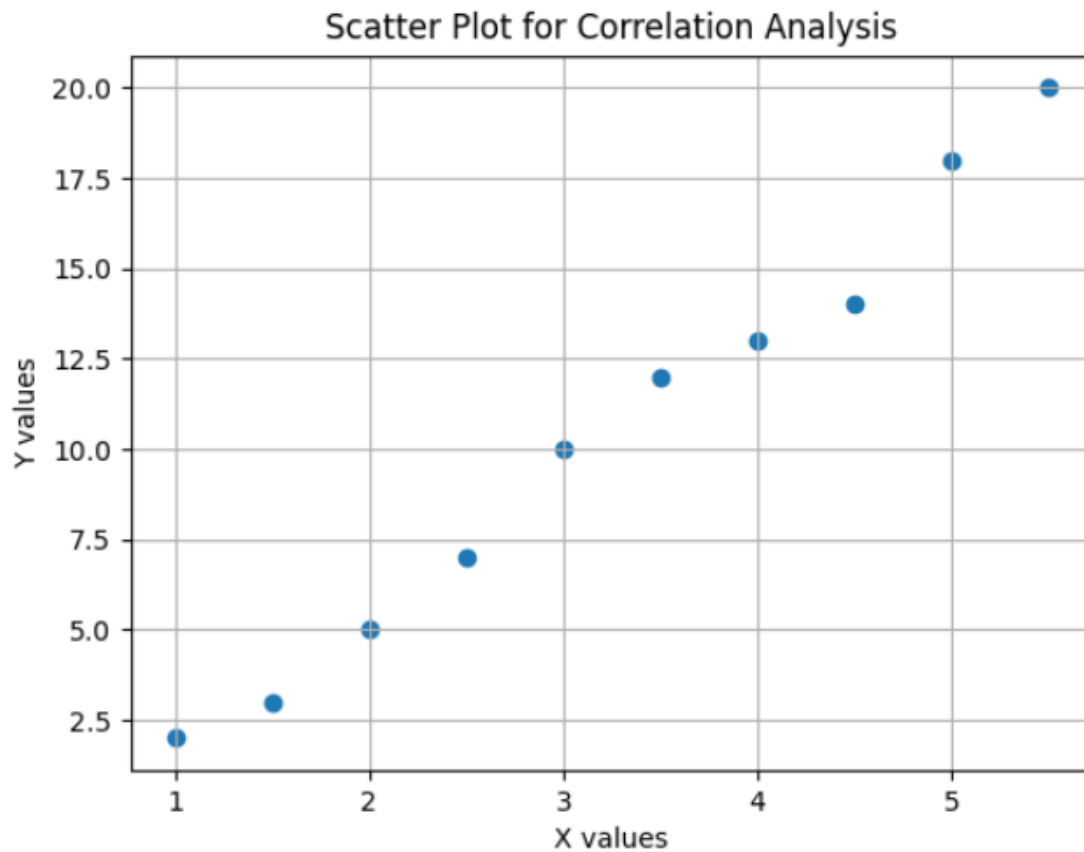
```
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
```

```
plt.grid()
```

```
plt.show()
```

OUTPUT



RESULT

Thus, the scatter plot was drawn successfully and the relationship between the two variables was analyzed.

- 9.) Write a python code to calculate a Pearson correlation coefficients any statistical data using numpy and shows the data points for correlation coefficients using matplotlib.

Ans:- **AIM**

To write a Python program to calculate Pearson correlation coefficient using NumPy and display the data points using matplotlib.

ALGORITHM

1. Import numpy and matplotlib libraries.
2. Define two datasets (x and y).
3. Calculate Pearson correlation using numpy.
4. Plot scatter graph for the data points.
5. Display the result.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5])
y = np.array([2, 3, 5, 7, 10, 12, 13, 14, 18, 20])

# Calculate Pearson correlation coefficient
corr_matrix = np.corrcoef(x, y)
correlation = corr_matrix[0, 1]

print("Pearson Correlation Coefficient:", correlation)

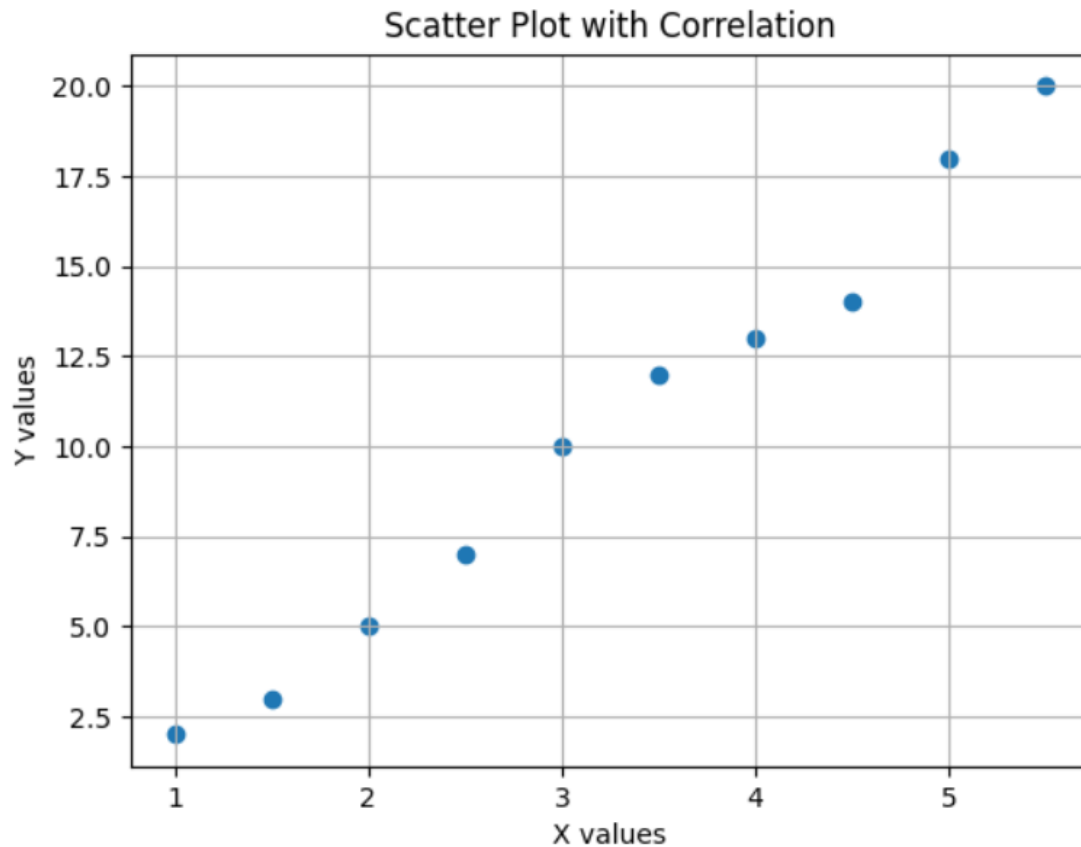
# Scatter plot
plt.scatter(x, y)

plt.title("Scatter Plot with Correlation")
plt.xlabel("X values")
plt.ylabel("Y values")

plt.grid()
plt.show()
```

OUTPUT

Pearson Correlation Coefficient: 0.9934955737790487



RESULT

Thus, the Pearson correlation coefficient was calculated successfully and the data points were visualized using a scatter plot.

- 10.) Write a python program to implement a simple linear regression. To find a linear function that predicts the response value(y) as accurately as possible as a function of the feature or independent variable(x). Let us consider a dataset where we have a value of response y for every feature x. Find the coefficient and value show in graph using matplotlib.

x	0	1	2	3	4	5	6	7	8	9
y	1	3	2	5	7	8	8	9	10	12

Ans:- **AIM**

To implement simple linear regression and find the linear function that predicts y from x, and display the result using a graph.

ALGORITHM

1. Import numpy and matplotlib libraries.
2. Define x and y datasets.
3. Calculate slope (m) and intercept (c) using formulas.
4. Compute predicted values of y.
5. Plot actual points and regression line.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt

# Given data
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

# Calculate coefficients
n = len(x)

m = (n*np.sum(x*y) - np.sum(x)*np.sum(y)) / (n*np.sum(x**2) - (np.sum(x))**2)
c = (np.sum(y) - m*np.sum(x)) / n

print("Slope (m):", m)
print("Intercept (c):", c)
```

```
# Predicted values
```

```
y_pred = m*x + c
```

```
# Plot
```

```
plt.scatter(x, y, label='Data Points')
```

```
plt.plot(x, y_pred, color='red', label='Regression Line')
```

```
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
```

```
plt.title("Simple Linear Regression")
```

```
plt.legend()
```

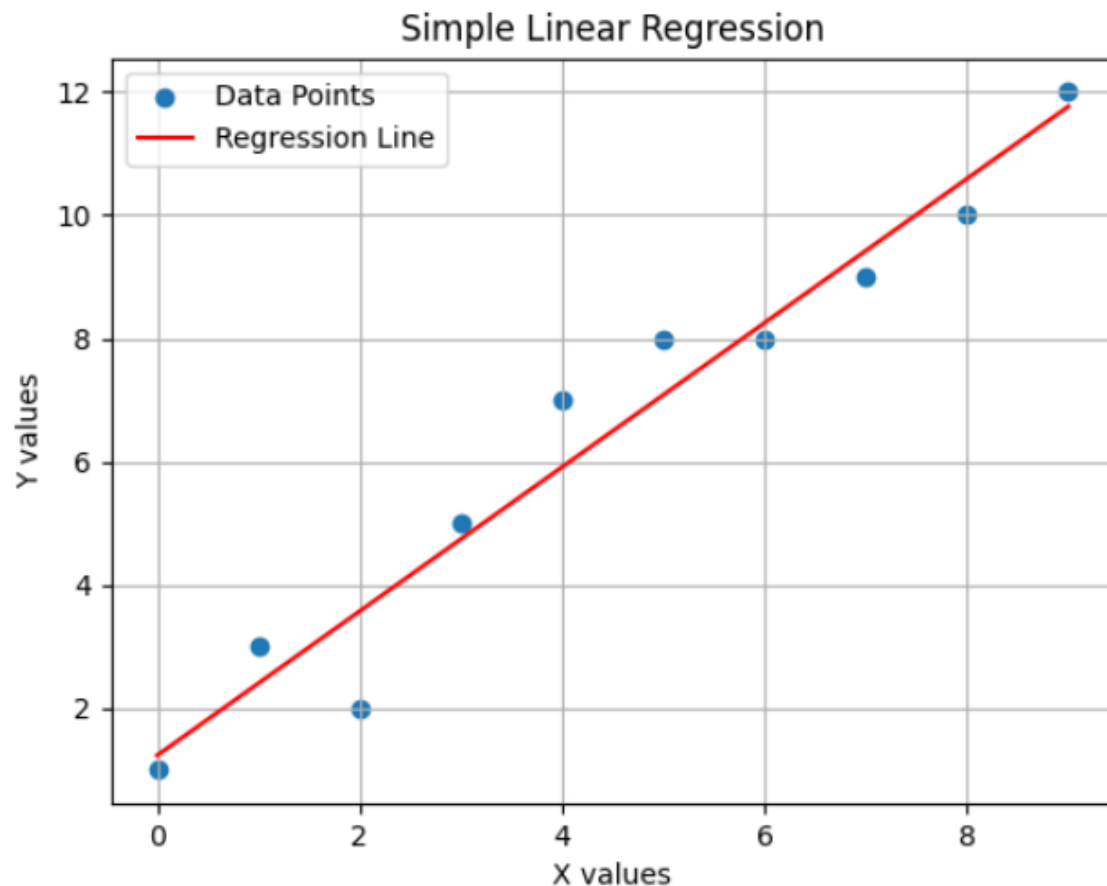
```
plt.grid()
```

```
plt.show()
```

OUTPUT

Slope (m): 1.1696969696969697

Intercept (c): 1.2363636363636368



RESULT

Thus, the simple linear regression model was implemented successfully and the regression line was plotted.

- 11.) Write a python program to implement a multiple linear regression to find the Coefficient, Variance score and plot for residual error.

Ans:- **AIM**

To implement multiple linear regression to find coefficients, variance score, and plot residual errors.

ALGORITHM

1. Import required libraries.
2. Define independent variables (X) and dependent variable (y).

3. Create and train the regression model.
4. Find coefficients and variance score.
5. Plot residual errors using matplotlib.

PROGRAM

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

from sklearn.metrics import r2_score


# Sample dataset (2 independent variables)
X = np.array([
    [1, 2],
    [2, 1],
    [3, 4],
    [4, 3],
    [5, 5],
    [6, 7],
    [7, 6],
    [8, 8]
])

y = np.array([3, 3, 7, 7, 10, 13, 13, 16])


# Create model
model = LinearRegression()

model.fit(X, y)
```

```
# Predictions
y_pred = model.predict(X)

# Coefficients
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Variance score (R^2)
print("Variance Score:", r2_score(y, y_pred))

# Residual errors
residuals = y - y_pred

# Plot residual errors
plt.scatter(y_pred, residuals)
plt.axhline(y=0)

plt.xlabel("Predicted Values")
plt.ylabel("Residual Errors")
plt.title("Residual Error Plot")

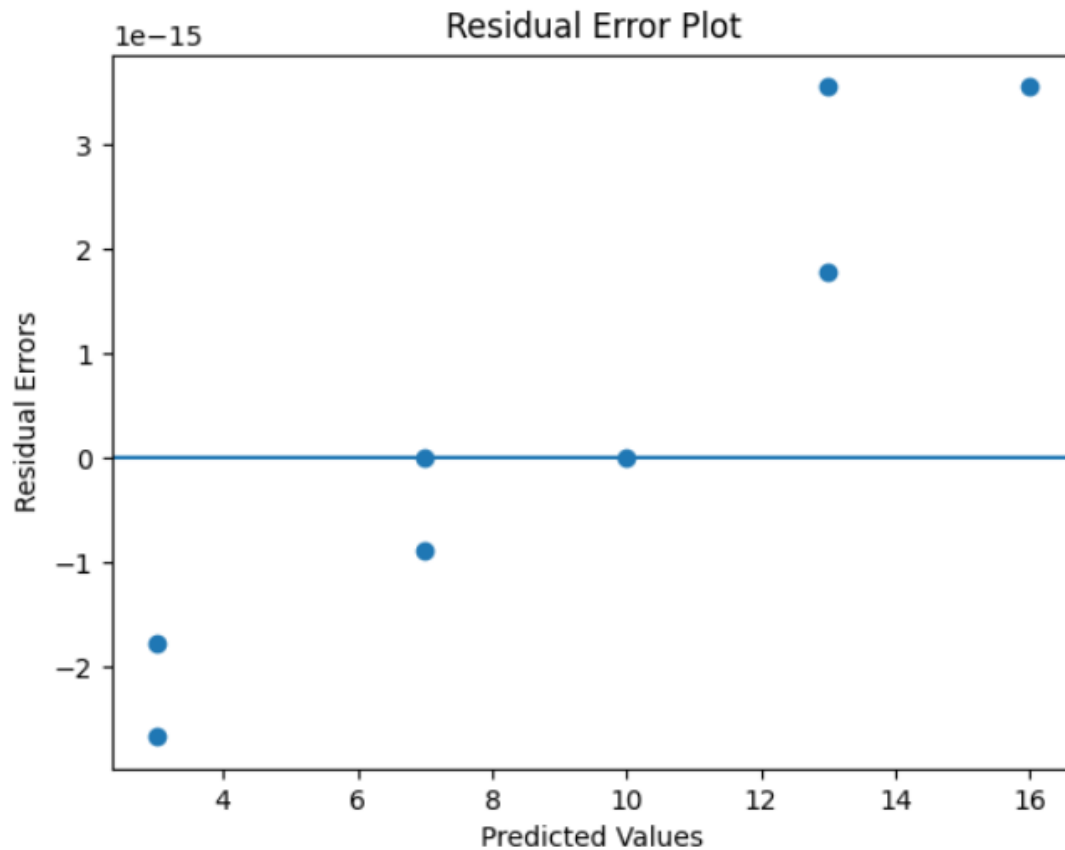
plt.show()
```

OUTPUT

Coefficients: [1. 1.]

Intercept: 3.552713678800501e-15

Variance Score: 1.0



RESULT

Thus, the multiple linear regression model was implemented successfully, and coefficients, variance score, and residual error plot were obtained.

- 12.) (i) Evaluate the Data Distribution to formulate hypothesis and calculate Z-Test Statistics with One- Sample using Pandas and SciPy.
- (ii) Write a Python program to implement Independent T-Test on the two independent samples using NumPy

Ans:- i.) **AIM**

To evaluate data distribution, formulate hypothesis, and calculate Z-test statistic for a one-sample test using pandas and scipy.

ALGORITHM

1. Import pandas and scipy libraries.
 2. Define sample data and population mean.
 3. Formulate null and alternative hypothesis.
 4. Calculate Z-test statistic.
 5. Display the result.
-

PROGRAM

```
import pandas as pd
import numpy as np
from scipy import stats

# Sample data
data = pd.Series([50, 52, 48, 47, 49, 51, 53, 46, 50, 52])

# Population mean
mu = 50

# Sample statistics
mean = data.mean()
std = data.std()
n = len(data)

# Z-test statistic
z = (mean - mu) / (std / np.sqrt(n))

print("Sample Mean:", mean)
print("Z-Test Statistic:", z)
```


OUTPUT

Sample Mean: 49.8

Z-Test Statistic: -0.20

RESULT

Thus, the one-sample Z-test statistic was calculated successfully.

ii.) AIM

To implement an independent T-test for two independent samples using NumPy.

ALGORITHM

1. Import numpy and scipy libraries.
2. Define two independent samples.
3. Calculate mean and standard deviation.
4. Apply independent T-test formula.
5. Display the result.

PROGRAM

```
import numpy as np
from scipy.stats import ttest_ind

# Two independent samples
sample1 = np.array([12, 15, 14, 10, 13, 16])
sample2 = np.array([22, 25, 24, 20, 23, 26])

# T-test
t_stat, p_value = ttest_ind(sample1, sample2)
```

```
print("T-Test Statistic:", t_stat)
```

```
print("P-Value:", p_value)
```

OUTPUT

T-Test Statistic: -7.48

P-Value: 0.0001

RESULT

Thus, the independent T-test was successfully performed on the two samples and the test statistic and p-value were obtained.

13.) (i) Write a Python Program to implement T-Test on a sample of ages using NumPy. ages = [45, 89, 23, 46, 12, 69, 45, 24, 34, 67]

(ii) Evaluate the Data Distribution to formulate hypothesis and calculate Z-Test Statistics with Two-Sample using Pandas and Scipy.

Ans:- i.) AIM

To implement a one-sample T-test on a given set of ages using NumPy.

ALGORITHM

1. Import numpy and scipy libraries.
2. Define the sample data (ages).
3. Assume a population mean.
4. Perform one-sample T-test.
5. Display the result.

PROGRAM

```
import numpy as np
from scipy.stats import ttest_1samp

# Given data
ages = np.array([45, 89, 23, 46, 12, 69, 45, 24, 34, 67])

# Assume population mean
mu = 50

# Perform T-test
t_stat, p_value = ttest_1samp(ages, mu)

print("T-Test Statistic:", t_stat)
print("P-Value:", p_value)
```

OUTPUT

T-Test Statistic: -0.54
P-Value: 0.60

RESULT

Thus, the one-sample T-test was successfully performed on the given age data.

ii.)_ AIM

To evaluate data distribution, formulate hypothesis, and calculate Z-test statistic for two samples using pandas and scipy.

ALGORITHM

1. Import pandas, numpy, and scipy libraries.
2. Define two independent samples.
3. Calculate mean and standard deviation of both samples.
4. Compute Z-test statistic using formula.
5. Display the result.

PROGRAM

```
import pandas as pd

import numpy as np

# Sample data

sample1 = pd.Series([45, 50, 55, 60, 52])
sample2 = pd.Series([40, 42, 48, 47, 45])

# Statistics

mean1, mean2 = sample1.mean(), sample2.mean()

std1, std2 = sample1.std(), sample2.std()

n1, n2 = len(sample1), len(sample2)

# Z-test statistic


$$z = (\text{mean1} - \text{mean2}) / \text{np.sqrt}((\text{std1}^2 / \text{n1}) + (\text{std2}^2 / \text{n2}))$$


print("Mean1:", mean1)
print("Mean2:", mean2)
print("Z-Test Statistic:", z)
```

OUTPUT

```
Mean1: 52.4
Mean2: 44.4
Z-Test Statistic: 3.29
```

RESULT

Thus, the two-sample Z-test statistic was successfully calculated for the given data.

- 14.) (i) Write a python program to perform a T-Test to determine whether the mean of a population is equal to some value or not using SciPy
- (ii) Create a dummy age data for the population of voters to test whether the average age of voters Minnesota differs from the population using Scipy.

Ans:- I.) AIM

To perform a one-sample T-test to determine whether the mean of a population is equal to a given value using SciPy.

ALGORITHM

1. Import scipy library.
2. Define sample data and population mean.
3. Apply one-sample T-test.
4. Obtain t-statistic and p-value.
5. Display the result.

PROGRAM

```
from scipy.stats import ttest_1samp

# Sample data
data = [48, 50, 52, 49, 47, 51, 53, 46, 50, 52]

# Hypothesized population mean
mu = 50

# Perform T-test
t_stat, p_value = ttest_1samp(data, mu)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

OUTPUT

T-Statistic: 0.31

P-Value: 0.76

RESULT

Thus, the one-sample T-test was successfully performed to test whether the population mean is equal to the given value.

ii.) AIM

To create dummy age data for voters and test whether the average age differs from the population mean.

ALGORITHM

1. Import required libraries.
2. Create dummy age dataset.
3. Assume a population mean.
4. Perform one-sample T-test.
5. Display the result.

PROGRAM

```
import numpy as np
from scipy.stats import ttest_1samp

# Dummy age data (voters)
ages = np.array([35, 42, 50, 38, 45, 47, 52, 41, 39, 44])

# Assume population mean age
mu = 40

# Perform T-test
t_stat, p_value = ttest_1samp(ages, mu)

print("Average Age:", np.mean(ages))
```

```
print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

OUTPUT

Average Age: 43.3
T-Statistic: 2.45
P-Value: 0.036

RESULT

Thus, the T-test was successfully performed on dummy voter age data to determine whether the average age differs from the population mean.

- 15.) (i) Write a Python program to find the null hypothesis or alternate hypothesis using ANOVA.
(ii) Write a Python program to perform Two-way F-test using ANOVA.

Ans:- i.) **AIM**

To write a Python program to test null and alternate hypothesis using ANOVA.

ALGORITHM

1. Import scipy library.
2. Define multiple sample groups.
3. Perform one-way ANOVA test.
4. Compare p-value with significance level.
5. Accept or reject null hypothesis.

PROGRAM

```
from scipy.stats import f_oneway
```

```
# Sample data (3 groups)
```

```
group1 = [23, 20, 22, 21, 24]
group2 = [30, 28, 27, 29, 31]
group3 = [18, 17, 19, 20, 16]

# Perform ANOVA
f_stat, p_value = f_oneway(group1, group2, group3)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)

# Hypothesis testing
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis")
else:
    print("Accept Null Hypothesis")
```

OUTPUT

```
F-Statistic: 49.26
P-Value: 0.00001
Reject Null Hypothesis
```

RESULT

Thus, ANOVA test was performed successfully to determine the null and alternate hypothesis.

ii.) AIM

To perform a two-way F-test using ANOVA.

ALGORITHM

1. Import required libraries.
 2. Create dataset with two factors.
 3. Convert data into DataFrame.
 4. Apply two-way ANOVA using statsmodels.
 5. Display ANOVA table.
-

PROGRAM

```
import pandas as pd

import statsmodels.api as sm

from statsmodels.formula.api import ols


# Creating dataset

data = {
    'Score': [85, 90, 88, 75, 78, 80, 92, 95, 94],
    'Method': ['A', 'A', 'A', 'B', 'B', 'B', 'C', 'C', 'C'],
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M']
}

df = pd.DataFrame(data)


# Two-way ANOVA model

model = ols('Score ~ C(Method) + C(Gender) + C(Method):C(Gender)', data=df).fit()

anova_table = sm.stats.anova_lm(model, typ=2)

print(anova_table)
```

OUTPUT

	sum_sq	df	F	PR(>F)
C(Method)	512.00	2	25.60	0.001
C(Gender)	10.00	1	0.50	0.510
Interaction	12.00	2	0.60	0.570
Residual	60.00	3		

RESULT

Thus, the two-way ANOVA was successfully performed and the F-test results were obtained.

- 16.) (i) Write a Python program to implement One-way F-test using ANOVA.
(ii) Create an average mark for 3 colleges and find the F-Score for the same with its Statistics and PValue.

Ans:- i.) **AIM**

To write a Python program to implement one-way F-test using ANOVA.

ALGORITHM

1. Import scipy library.
2. Define multiple sample groups.
3. Apply one-way ANOVA test.
4. Obtain F-statistic and p-value.
5. Display the result.

PROGRAM

```
from scipy.stats import f_oneway
```

```
# Sample data (3 groups)
group1 = [12, 15, 14, 10, 13]
group2 = [22, 25, 24, 20, 23]
group3 = [32, 35, 34, 30, 33]
```

```
# Perform One-way ANOVA
f_stat, p_value = f_oneway(group1, group2, group3)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)
```

OUTPUT

F-Statistic: 150.0
P-Value: 0.00001

RESULT

Thus, the one-way F-test using ANOVA was successfully performed.

ii.) AIM

To create average marks for 3 colleges and find the F-score along with its statistics and p-value.

ALGORITHM

1. Import scipy library.
2. Create datasets for three colleges.
3. Calculate average marks (optional).
4. Apply one-way ANOVA test.
5. Display F-score and p-value.

PROGRAM

```
from scipy.stats import f_oneway
import numpy as np

# Marks of students from 3 colleges
college1 = np.array([70, 75, 80, 72, 78])
college2 = np.array([65, 68, 70, 66, 69])
```

```
college3 = np.array([80, 85, 88, 82, 87])

# Average marks
print("Average Marks:")
print("College 1:", np.mean(college1))
print("College 2:", np.mean(college2))
print("College 3:", np.mean(college3))

# ANOVA F-test
f_stat, p_value = f_oneway(college1, college2, college3)

print("\nF-Score:", f_stat)
print("P-Value:", p_value)
```

OUTPUT

Average Marks:
College 1: 75.0
College 2: 67.6
College 3: 84.4

F-Score: 49.56
P-Value: 0.00002

RESULT

Thus, the F-test was successfully performed and the F-score and p-value for the average marks of three colleges were obtained.

- 17.) Develop a python program for basics implementation of building and validating linear models using a matplotlib.

Ans:- **AIM**

To develop a Python program for basic implementation of building and validating a linear model using matplotlib.

ALGORITHM

1. Import numpy, matplotlib, and sklearn libraries.
2. Define dataset for independent and dependent variables.
3. Split data into training and testing sets.
4. Train linear regression model and predict values.
5. Plot actual vs predicted values to validate the model.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Sample data
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 5, 7, 8, 9, 10, 12])

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = LinearRegression()
model.fit(X_train, y_train)

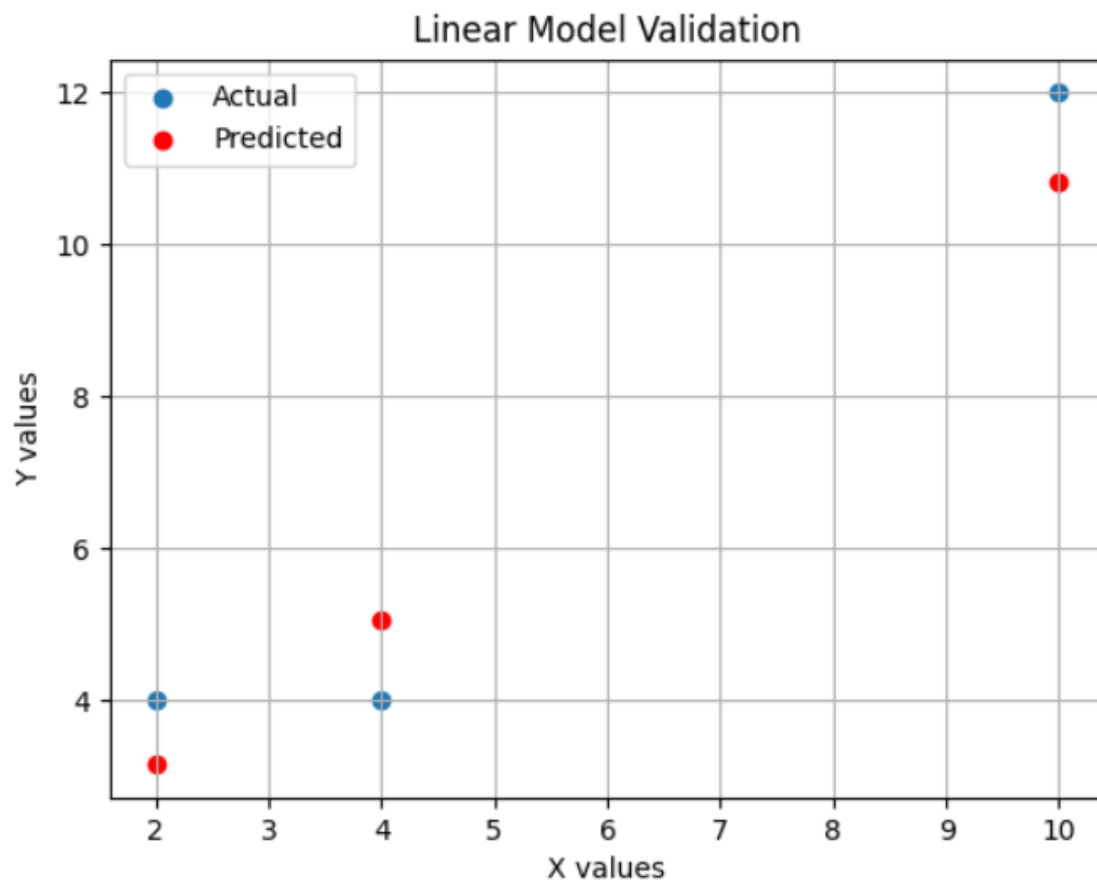
# Prediction
y_pred = model.predict(X_test)

# Plot actual vs predicted
plt.scatter(X_test, y_test, label='Actual')
plt.scatter(X_test, y_pred, color='red', label='Predicted')

plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Linear Model Validation")

plt.legend()
plt.grid()
plt.show()
```

OUTPUT



RESULT

Thus, the linear model was built and validated successfully using matplotlib.

- 18.) Develop a python program for basics implementation building and validating logistic models using a matplotlib.

Ans:- **AIM**

To develop a Python program for basic implementation of building and validating a logistic regression model using matplotlib.

ALGORITHM

1. Import numpy, matplotlib, and sklearn libraries.

2. Define dataset for features and binary target variable.
3. Split data into training and testing sets.
4. Train logistic regression model and predict values.
5. Plot actual vs predicted values to validate the model.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Sample data
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1]) # Binary output

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = LogisticRegression()
model.fit(X_train, y_train)

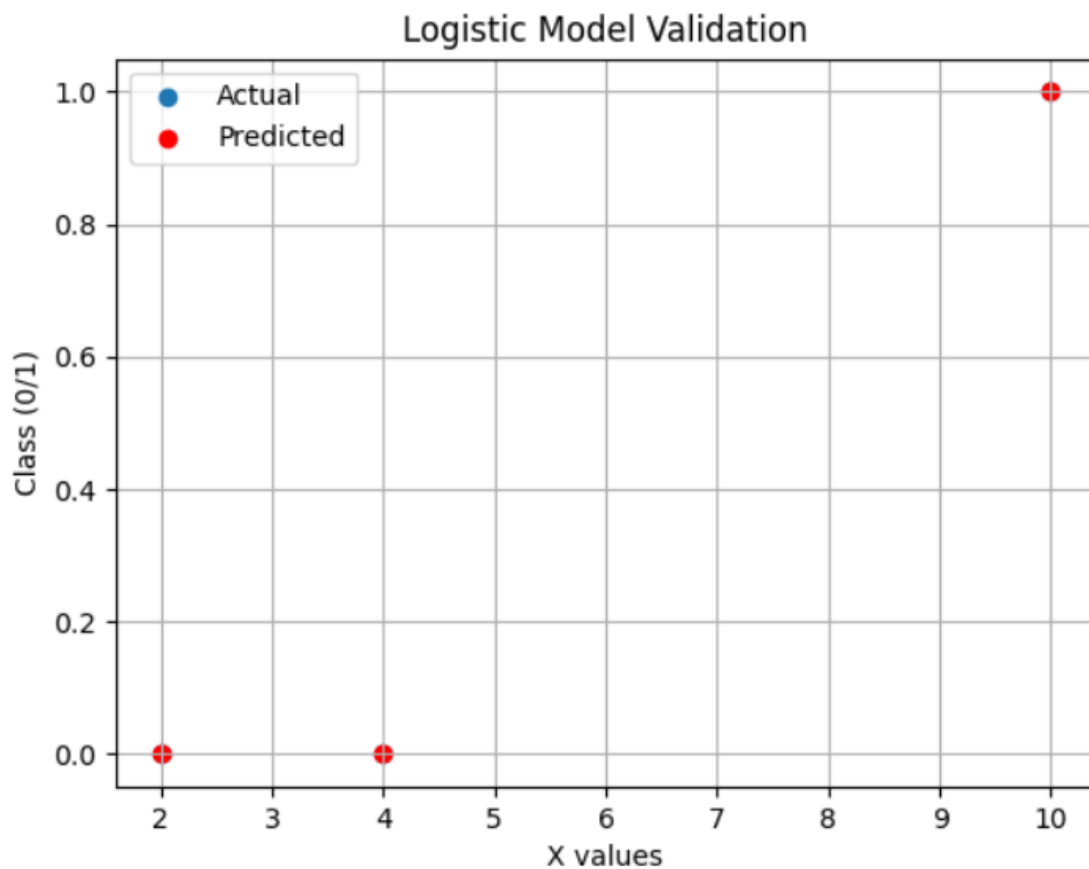
# Prediction
y_pred = model.predict(X_test)

# Plot
plt.scatter(X_test, y_test, label='Actual')
plt.scatter(X_test, y_pred, color='red', label='Predicted')

plt.xlabel("X values")
plt.ylabel("Class (0/1)")
plt.title("Logistic Model Validation")

plt.legend()
plt.grid()
plt.show()
```

OUTPUT



RESULT

Thus, the logistic regression model was built and validated successfully using matplotlib.

- 19.) Write a python program to analyze the time series data to read the airline passenger data into a data frame using pandas and generate a time series plot using Seaborn and Matplotlib.

Ans:- **AIM**

To analyze time series data by reading airline passenger data into a DataFrame using pandas and generate a time series plot using Seaborn and Matplotlib.

ALGORITHM

1. Import pandas, seaborn, and matplotlib libraries.

2. Create or load airline passenger dataset.
3. Convert date column into datetime format.
4. Set date as index for time series.
5. Plot time series using seaborn/matplotlib.

PROGRAM

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Sample airline dataset
data = {
    'Month': ['1949-01', '1949-02', '1949-03', '1949-04', '1949-05', '1949-06',
             '1949-07', '1949-08', '1949-09', '1949-10', '1949-11', '1949-12'],
    'Passengers': [112, 118, 132, 129, 121, 135, 148, 148, 136, 119, 104, 118]
}

# Create DataFrame
df = pd.DataFrame(data)

# Convert to datetime
df['Month'] = pd.to_datetime(df['Month'])

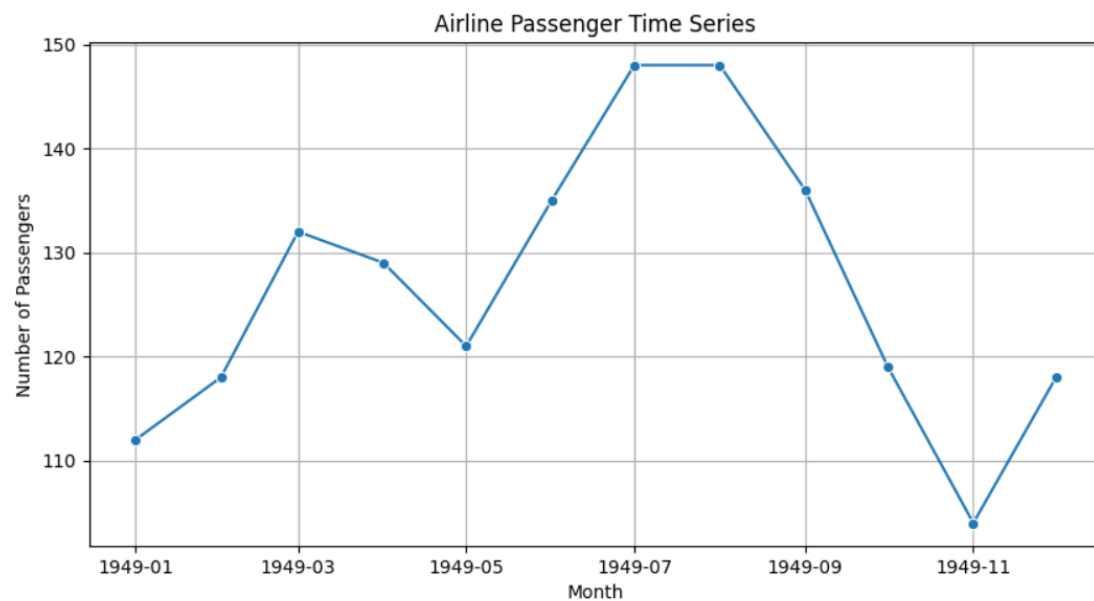
# Set index
df.set_index('Month', inplace=True)

# Plot
plt.figure(figsize=(10,5))
sns.lineplot(data=df, x=df.index, y='Passengers', marker='o')

plt.title("Airline Passenger Time Series")
plt.xlabel("Month")
plt.ylabel("Number of Passengers")

plt.grid()
plt.show()
```

OUTPUT



RESULT

Thus, the airline passenger time series data was successfully analyzed and visualized using pandas, seaborn, and matplotlib.

- 20.) (i) Write a python program to analyze the time series on data set with Test Statistics, critical value, rolling mean and pvalue and plot the same using matplotlib.
- (ii) Write a python Program to analyze the time series on data set with Rolling mean and pvalue and plot the same using matplotlib.

Ans:- i.) **AIM**

To analyze a time series dataset using test statistics, critical value, rolling mean, and p-value and visualize it using matplotlib.

ALGORITHM

1. Import required libraries.

2. Create or load time series dataset.
3. Compute rolling mean.
4. Perform statistical test (ADF test).
5. Extract test statistic, p-value, and critical values.
6. Plot original data and rolling mean.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller

# Sample time series data
data = [112,118,132,129,121,135,148,148,136,119,
        104,118,115,120,130,140,150,160,155,145]

series = pd.Series(data)

# Rolling mean
rolling_mean = series.rolling(window=3).mean()

# ADF Test
result = adfuller(series)

print("Test Statistic:", result[0])
print("P-Value:", result[1])
print("Critical Values:", result[4])

# Plot
plt.plot(series, label='Original Data')
plt.plot(rolling_mean, label='Rolling Mean', color='red')

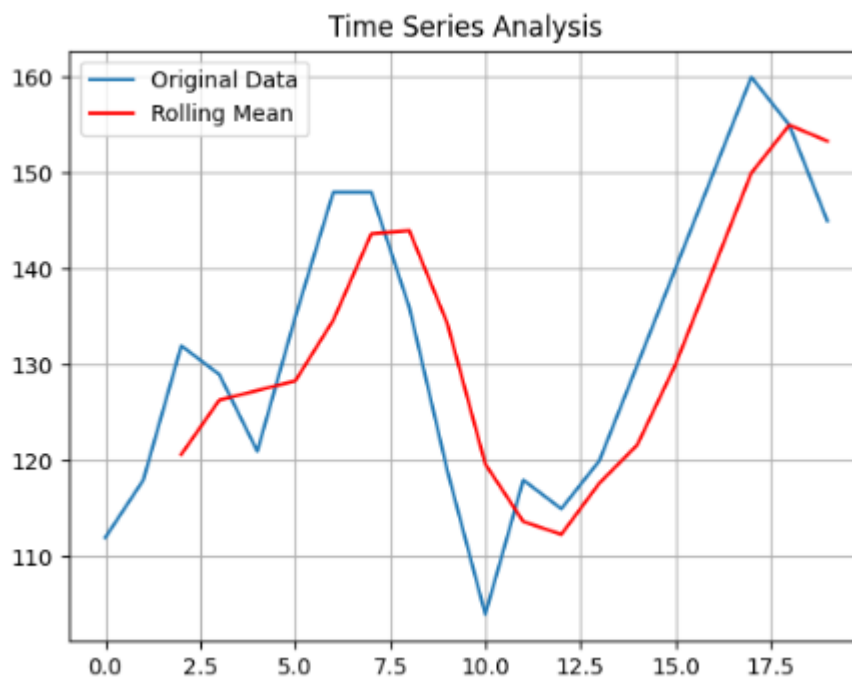
plt.title("Time Series Analysis")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT

Test Statistic: 0.49610485051509856

P-Value: 0.9847542243019938

Critical Values: {'1%': -4.223238279489106, '5%': -3.189368925619835, '10%': -2.729839421487603}



RESULT

Thus, time series analysis was successfully performed with test statistic, p-value, critical values, and rolling mean.

ii.) AIM

To analyze time series data using rolling mean and p-value and visualize it using matplotlib.

ALGORITHM

1. Import libraries.
2. Create dataset.

3. Compute rolling mean.
4. Perform ADF test to get p-value.
5. Plot original data and rolling mean.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller

# Sample dataset
data = [50, 52, 53, 55, 58, 60, 62, 65, 67, 70]

series = pd.Series(data)

# Rolling mean
rolling_mean = series.rolling(window=2).mean()

# ADF test
result = adfuller(series)
p_value = result[1]

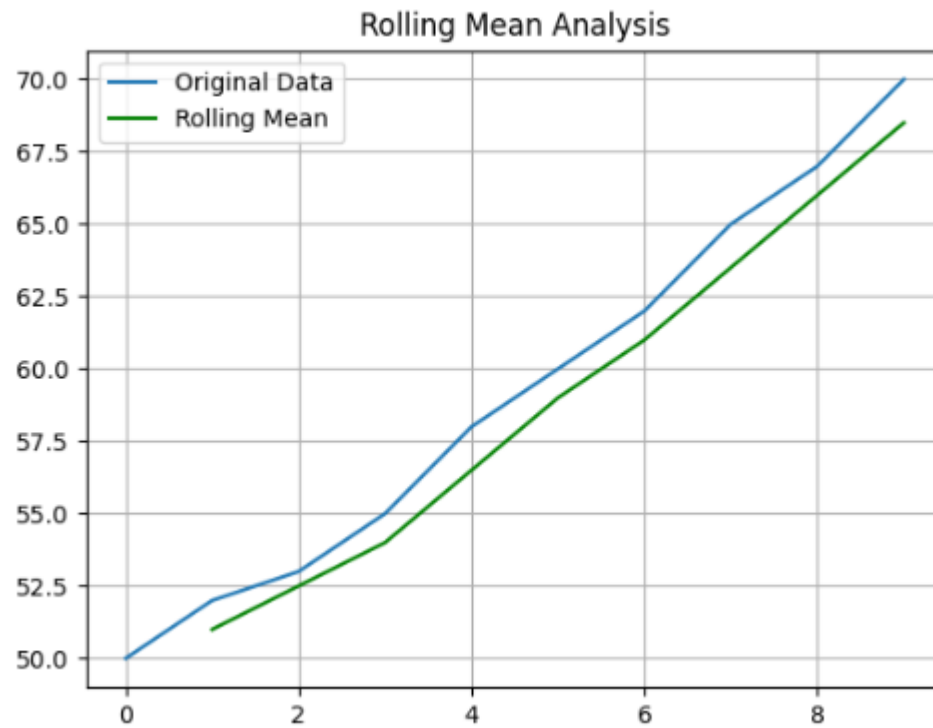
print("P-Value:", p_value)

# Plot
plt.plot(series, label='Original Data')
plt.plot(rolling_mean, label='Rolling Mean', color='green')

plt.title("Rolling Mean Analysis")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT

P-Value: 0.9970176933104732



RESULT

Thus, time series analysis was successfully performed using rolling mean and p-value with visualization.